

编程技巧手册 · 2026-09-08

BNCT GPU 蒙特卡罗项目 · 第 1 部分「今天用到的」 / 第 2 部分「明天用到的」 / 第 3 部分「数模比赛四天会用到的」

用法：不要通读。第 1 部分今晚扫一遍把名字对上号；第 2、3 部分明天上午看；第 5 部分速查表放手边随时查。

第 0 部分 · 今天到底做了什么（一页总览）

交付物	内容	判据（怎么知道它对）
git 仓库 本地 9 个提交	第 1-4 课全部代码入库，分主题拆成多个提交；远端 origin 已配（未 push）	<code>git log --oneline --stat</code> ：每条消息和它的文件清单对得上
mcstat.py	统计模块： <code>mean_se</code> / <code>dev_sigma</code> / <code>row</code> / <code>header</code> / <code>chi2_uniform</code> / <code>check</code>	<code>python mcstat.py</code> 两条自测通过，退出码 0
lesson04 改造	九行统计塌成三行；五处 <code>row+assert</code> 塌成 <code>check</code>	输出 26 行 = 26 行， 0 处不同
lesson03 改造	六处统计公式换成 <code>mcstat.mean_se</code> ；新增 csv 输出	输出 80 行 = 80 行， 0 处不同
第一张图	三个 case 的深度通量曲线 + 误差棒	轴有量+单位、有图例、误差棒放大 100 倍验过

一句话：今天学的不是「写新功能」，是「改一大片代码而行为一个字节不变」。这项技能在 11 月把 Python 翻成 C++、12 月翻成 CUDA 时，是你唯一能依靠的东西。

第 1 部分 · 今天用到的技巧（全）

1.1 git — 版本控制

心智模型：三个区

工作区 (文件夹里的文件)	暂存区 (待提交清单)	仓库 (快照序列 .git/)
--- git add ---->		
	--- git commit -->	
<----- git checkout (丢弃修改) ----		

暂存区存在的唯一理由：你一次改了五个文件，但它们属于两件不同的事。暂存区让你只把属于同一件事的那几个放进同一个提交。

一个提交 = 一件事。 提交说明写不成一句话，就说明该拆成两个提交。

一次性配置（一台机器只做一次）

```
git config --global user.name "WebbWuWU"
git config --global user.email "12345678+WebbWuWU@users.noreply.github.com"
git config --global init.defaultBranch main
git config --global core.quotepath false      # 让 git status 正常显示中文文件名
```

邮箱会写进每一个提交并在公开仓库里永久可见。 用 GitHub 的 noreply 地址可以既被正确归属、又不暴露真实邮箱。**已经写进历史的拔不出来**——要在第一个提交之前决定。

日常命令

命令	作用
<code>git status</code>	我现在在什么状态。 一天敲二十次 ，卡住先敲它，它会顺带告诉你下一步
<code>git add 文件...</code>	放进暂存区。 <code>git add .</code> 是全部，但第一次导入时别用
<code>git commit -m "说明"</code>	打快照。 没写 -m 会掉进 vim ：按 <code>Esc</code> ，输入 <code>:q!</code> ，回车逃出来
<code>git log --oneline</code>	看提交序列，一行一个
<code>git log --oneline --stat</code>	多打印每个提交改了哪些文件、加删各几行 ——今天新立的核对判据
<code>git diff</code>	工作区 vs 暂存区：我改了但还没 add 的
<code>git diff --staged</code>	暂存区 vs 上个快照：add 了但还没 commit 的
<code>git diff --numstat -- 文件</code>	只看「加几行删几行」，恢复文件时最快的核对方式
<code>git show <id>:<文件></code>	把某个历史版本的文件整个吐出来（今天用它恢复了 lesson03 的 114 行注释）
<code>git reset <id></code>	把分支指针搬回某个提交， 工作区文件一个不动 ，暂存区清空
<code>git checkout 文件</code>	把某个文件退回上次提交的样子（ 会丢弃你的修改 ）
<code>git remote add origin <url></code> <code>git remote set-url origin <url></code>	加 / 改远端地址。已存在时 <code>add</code> 会报 <code>remote origin already exists</code> ，那时候要用 <code>set-url</code>
<code>git push -u origin main</code>	推上去。 <code>-u</code> 只需第一次，之后 <code>git push</code> 就够

.gitignore

```
__pycache__/      # Python 字节码缓存，从 .py 自动生成的东西一律不进仓库
*.pyc
.vscode/          # 编辑器本地配置，是你这台机器的设置不是项目的一部分
*.png
before_*.txt      # 对拍用的中间产物
after_*.txt
```

必须在第一次 `git add` 之前写。 文件一旦被提交过一次，就永远留在历史里，后面再加进 `.gitignore` 也删不掉那份历史。

判据：写完敲 `git status`，输出里不该再出现 `__pycache__`。

今天踩到的三个 git 坑

现象	根因与对策
两个提交的消息和内容对不上	多行粘贴进 PowerShell 时命令错位。 对策： <code>git add ...</code> ； <code>git commit -m "..."</code> 写成一行，每次只粘一行
<code>error: unknown switch 'm'</code>	两行之间的换行被吃掉， <code>git add</code> 收到了下一行的 <code>-m</code> 。回显里 <code>spec.mdgit</code> 就是烟枪
一个提交顺手删了 124 行注释而判据没发现	「输出逐位不变」对注释免疫。 对策： 提交前扫一眼 <code>git log --stat</code> 的加删条形图，减号占大头就要起疑

1.2 Python 语言层

模块与 import

```
import mcstat                                # 用的时候写 mcstat.mean_se(...)
import matplotlib.pyplot as plt             # as 起别名; plt 是全世界的约定, 别改
```

为什么用 `import mcstat` 而不是 `from mcstat import *`: 三个月后你读到

`mcstat.chi2_uniform(...)`, 一眼知道它来自哪个文件; 写成 `chi2_uniform(...)` 就要翻回顶部去找。代码要能自证出处。

同目录下的 `.py` 文件直接 `import` 文件名 (不带 `.py`) 即可。import 之后目录里会多一个 `__pycache__`, 那是编译缓存, 已被 `.gitignore` 挡住。

函数抽象: 这是今天最核心的一课

```
def mean_se(s, sq, n):
    """由  $\Sigma x$ ,  $\Sigma x^2$ ,  $n$  算样本均值、贝塞尔方差、均值标准误。"""
    assert n > 1, "n取值不对, n=%d" % n
    mean = s / n
    var = n / (n - 1) * (sq / n - mean ** 2)
    assert var >= 0, "var范围出错, var=%.6f" % var
    se = math.sqrt(var / n)
    return mean, var, se
```

「调用方负责攒, 函数负责算」—— 函数抽象的全部内容。

`mean_se` 从头到尾看不到任何一个样本值 `x`, 它只收三个已经累加好的标量。这不是风格选择:

- ① 省内存 ($N=1e6$ 的数组是 8 MB, 而这些内存毫无用处)
 - ② 这是**在线算法**, 不需要预先知道 N , 随时可以问「现在的均值和误差是多少」
 - ③ **GPU 上没有别的选择**: $1e9$ 条历史的样本存不下, 而 `atomicAdd` 能做的恰恰只有「往一个标量上加」。
- 今天写的这个签名, 和三个月后 **CUDA kernel** 里那份是同一个。

返回多个值 / 元组解包

```
return mean, var, se                        # 实际上是打包成一个元组
mean, var, se = mean_se(s, sq, n)          # 调用方一次拆开
_, _, err = mean_se(s, sq, n)              # 用不上的返回值, Python 习惯写 _
for path, label in CASES:                  # 循环里同时拆开元组, 同一个机制
```

默认参数

```
def check(name, meas, ref, se, k=3.0):      # 不传 k 就是 3.0
def print_flux_table(..., csv_path=None):   # 不传就不写 csv
```

默认参数的真正价值: 加功能而不破坏已有调用。给 `print_flux_table` 加 `csv_path=None` 之后, 原来三处调用一个字不改也照样能跑。

`k=3.0` 则把「判据强度」变成一个**可见的参数**, 而不是散落各处的魔法数字 3——第 9 课里程碑 1 要求「 1σ 内重合」, 那时候直接 `k=1`。

docstring

```
def row(name, meas, ref, se):  
    """打印表中每一行。"""      # 必须是函数体第一条语句，三引号
```

注释在运行时被丢弃；docstring 是存在函数对象里的数据（函数.__doc__），VS Code 悬停能显示，help() 能打出来。注释写给读源码的人，docstring 写给用这个函数的人。

```
if __name__ == "__main__":
```

```
    if __name__ == "__main__":  
        # 只有 python mcstat.py 直接跑才执行；被 import 时不执行  
        ...自测代码...
```

怎么运行	__name__ 的值
python mcstat.py	"__main__"
别处 import mcstat	"mcstat"

不加这个 guard，import 会执行模块顶层的所有语句，自测输出会混进调用方的输出里，对拍立刻变脏。

assert 的两个位置

```
assert 条件, "消息"      # 逗号分开，两个位置  
assert "mu_lab无定义"    # ← 错！断言了一个非空字符串，恒真，永远不会失败
```

assert 要挡在会炸的操作前面，不是全堆到函数开头。

assert n>1 挡的是除以 n-1；assert var>=0 挡的是 sqrt。所以正确顺序是：

docstring → assert n>1 → mean → var → assert var>=0 → se → return

今天新立的一条：写完一条 assert，问「恰好等于边界值的时候，被我放行的那个值真的能跑吗？」

assert n>=1 放行了 n=1，而 n/(n-1) 在 n=1 时必然除零。判据比被判的代码更容易错，因为没人去判判据。

列表

```
z = []          # 空列表  
z.append(3.5)   # 加到末尾  
len(z)         # 长度  
z[0]           # 第一个（下标从 0 开始）  
z[-1]          # 最后一个
```

切片 L[起:止]，起含、止不含：

```
L = ['表头', '数据1', '数据2', '数据3']  
L[1:]      → ['数据1', '数据2', '数据3']    # 从第 1 个到末尾（跳过表头就用这个）  
L[:2]      → ['表头', '数据1']  
L[1:3]     → ['数据1', '数据2']
```

列表推导式

```
[e * ERR_SCALE for e in err]
# 读法：对 err 里的每一个 e，算出 e*ERR_SCALE，组成新列表
```

列表不能直接乘数：

`[1,2] * 3` → `[1,2,1,2,1,2]`（重复三遍，不是每个元素乘 3）

`[1,2] * 1.0` → **`TypeError: can't multiply sequence by non-int of type 'float'`**

乘整数不报错但意思完全不对；乘浮点当场报错。今天你运气好，写的是 `1.0`。

字符串

```
" abc\n".strip()      → "abc"          # 去首尾空白，主要是干掉结尾的 \n
"1,2,3".split(",")    → ['1','2','3'] # 按逗号切成列表
float("0.05")         → 0.05           # 字符串不是数，不转不能算不能画

"% .6f" % x           # 6 位小数
"% .2e" % x           # 2 位有效数字的科学计数法
"%-12s" % s           # 左对齐占 12 格
"%+10.2f" % x         # 带正负号
"%%"                 # 字面百分号

"$\\Sigma_t$"         # 反斜杠在字符串里要写两个
r"$\Sigma_t$"         # 或者在引号前加 r (raw string)，等价
```

1.3 文件读写

```
with open(path, "w", encoding="utf-8") as f:
    f.write("k,z_lo,z_hi,z_mid,phi,err,R\n")      # write 不会替你换行
    f.write("%d,%.6f,%.8e\n" % (k, z, phi))

with open(path, "r", encoding="utf-8") as f:
    lines = f.readlines()      # 整个文件读成列表，每个元素是一行（含结尾 \n）
```

写法	为什么
<code>with ... as f:</code>	出了缩进块 自动关文件 ，哪怕中途抛异常。不关的话数据可能还卡在缓冲区没落盘
<code>"r" / "w" / "a"</code>	读 / 写（ 会清空原文件 ） / 追加
<code>encoding="utf-8"</code>	不写的话 Windows 默认 GBK，中文迟早乱码。 Windows 上永远显式写
文件里用 <code>%.8e</code>	屏幕是给人看的，文件是给下一个程序吃的。 精度别在中间环节丢掉

1.4 matplotlib

两层容器

Figure (画布) — 一整张图片, 存成 png 的就是它
└─ Axes (坐标系) — 一套 x 轴 y 轴和画在里面的东西

```
fig, ax = plt.subplots(figsize=(7, 5)) # 单位是英寸
```

网上大量教程用 `plt.plot()` / `plt.xlabel()` (pyplot 接口), 不要学那套。它偷偷操作「当前那张图」, 画一张时没区别, 但第 9 课 OpenMC 对拍要画双子图 (曲线 + 残差) 时会立刻乱套。`ax.xxx` 这套从一开始就说清楚画在哪个坐标系里。

今天用到的全部调用

```
ax.errorbar(x=z, y=phi, yerr=[e*S for e in err], label=label,
            marker="o", markersize=3, capsize=2, linewidth=1)
ax.set_xlabel("depth (cm)")
ax.set_ylabel("flux (cm-2 per source neutron)")
ax.set_title("...")
ax.legend() # 自动收集所有带 label= 的曲线
ax.grid(True, alpha=0.3) # alpha 是透明度
ax.set_yscale("log") # 纵轴换成对数

fig.savefig("x.png", dpi=150, bbox_inches="tight")
plt.show()
```

参数	作用
<code>yerr=</code>	误差棒半长, 你的 <code>err</code> 就是 1σ
<code>label=</code>	图例文字。拼成 <code>lable</code> 会报 <code>unexpected keyword argument</code>
<code>marker="o"</code>	数据点形状: <code>o</code> 圆、 <code>s</code> 方、 <code>^</code> 三角
<code>capsize=2</code>	误差棒两端小横杠的长度, 0 就没有横杠
<code>dpi=150</code>	每英寸像素数。7 英寸宽 $\times 150 = 1050$ 像素
<code>bbox_inches="tight"</code>	裁掉四周白边, 不加的话轴标签常被切掉半截
<code>\$...\$</code>	数学模式 (LaTeX 语法), <code>cm⁻²</code> $\rightarrow$ <code>cm⁻²</code> 。matplotlib 内置, 不用装 LaTeX

三个坑

- ① **savefig 必须在 show 之前**。`plt.show()` 的窗口关掉之后画布被清空, 再 `savefig` 得到一张全白的 png, 而且不报任何错。
- ② **中文标签变方块**。默认字体没有中文。标签一律用英文, 别去折腾字体。
- ③ **误差棒看不见 $\neq$ 没画上**。把 `yerr` 临时乘 100 跑一次, 看它变不变大。没见过它长出来的误差棒, 不算画上了。

● 对数轴的底数陷阱（今天亲自踩了）

`ax.set_yscale("log")` 用的是以 10 为底。

指数衰减 $\phi \propto e^{-\Sigma t \cdot z}$:

- **ln 坐标下斜率** = $-\Sigma t = -1.0$ （这才是物理）
- **log₁₀ 坐标下斜率** = $-\Sigma t \times \log_{10} e = -1.0 \times 0.4343 = -0.434$ （容易读成 -1/2）

幂律的斜率换底不变，指数衰减的斜率换底要乘 0.4343。

所以看见「斜率 -1/2」先问一句：这是半对数（只 y 取 log）还是双对数（x、y 都取 log）？

白板题⑥里 B-10 的 $1/v$ 在双对数上斜率 -1/2，那是真正的幂指数；今天这个 -1/2 是底数造成的假象。两个 -1/2 毫无关系。

要让斜率直接等于 $-\Sigma t$: `ax.plot(z, [math.log(p) for p in phi])`，自己取自然对数。

1.5 环境与工具

conda

```
conda activate openmc          # 激活环境
conda install -c conda-forge matplotlib  # 用和 openmc 同一个源，避免依赖打架
conda init powershell          # 让终端自动加载 conda（省掉每次手敲 hook）
python -c "import matplotlib; print(matplotlib.__version__)"  # 查包
```

查包必须在你真正要用的那个环境里查。今天在普通 cmd（base 环境）里查到 matplotlib 3.10.7，而 openmc 环境里根本没装。

看提示符前面有没有（openmc）。在错的环境里验出来的通过，等于没验。

PowerShell

写法	说明
命令A; 命令B	一行两条，今天用它绕开了多行粘贴的换行丢失问题
<code>python x.py > out.txt</code>	把 stdout 重定向到文件。⚠️ PowerShell 5.1 写出来的是 UTF-16LE，不是 UTF-8
<code>Compare-Object (Get-Content a) (Get-Content b)</code>	逐行比两个文件，无输出 = 完全相同。Get-Content 认 BOM，不受编码影响
<code>dir</code> 通配符	列文件
<code>Set-ExecutionPolicy RemoteSigned -Scope CurrentUser</code>	允许运行脚本（conda hook 要用）

● 判据本身失效的活标本：原计划用 `git diff --no-index before after` 做对拍判据，但 git 把 UTF-16 文件当二进制，只会输出一句 `Binary files ... differ` —— 只说「不一样」，不说哪一行、哪个数、差多少。

「我的判据本身能不能工作」也需要验一次。

python -u 与块缓冲

```
python -u lesson03_tally.py > out.txt      # -u = unbuffered , 文件一行行长出来
```

不加 `-u` 时，`stdout` 一旦被重定向就从行缓冲变成**块缓冲（8 KB）**：屏幕一片空白、文件长时间 0 字节，**看着像死机**。另一个判断办法：任务管理器看 `python.exe` 的 CPU，一个核满载 = 在算。

第 9 课跑 OpenMC、第 13 课跑 GPU kernel 时，「它是在算还是挂了」这个判断你会天天做。

VS Code

功能	用法
源代码管理面板	左侧第三个图标（分支形状）→ 点文件 → 左右并排看 diff ，左旧右新。比终端好用得多
问题面板	<code>Ctrl+Shift+M</code> ，按行号排序。 第一条是真的，其余当噪声
F2 重命名符号	选中变量名按 F2，作用域内所有引用一起改， 不可能漏 （今天 <code>counts / counts2</code> 漏改一半就是没用它）
状态栏行列号	光标点到行首非空字符，看「行 4, 列 6」—— 列号减 1 就是空格数 ，查缩进最快
终端分页器	<code>git diff</code> 会进分页器： 空格翻页，q 退出 （不知道按 q 会以为卡死）

Python 交互式解释器（今天该拿走的第三个工具）

```
> python      # 提示符变成 >>>
>>> L = ['a','b','c']
>>> L[1:]
['b', 'c']
>>> exit()
```

任何「这么写会得到什么」的疑问，在这里**三秒钟有答案**，不用改文件、不用跑整个程序。切片取到哪、函数返回什么、字符串怎么切——都来这里试两行。

1.6 调试：今天见过的 8 个报错

读 traceback 的规矩

最后一行告诉你「是什么错」；从上往下数第一个属于你自己文件的帧告诉你「在哪改」。
中间所有带 `site-packages` 的帧都是库的内部，看都不用看。

报错	真正的意思 / 今天的实例
<code>IndentationError: unindent does not match any outer indentation level</code>	缩进不一致。docstring 用了 5 个空格、其余用 4 个。 块里第一条语句定义了整个块的缩进
<code>SyntaxError</code> 而且后面每个 <code>def</code> 都标红	括号/引号没闭合，Python 认为这一行还没写完，一直往下读。 只看第一个标红处，后面全是连锁反应
<code>TypeError: 'module' object is not callable</code>	<code>mcstat(...)</code> —— 把模块当函数调用了，应该是 <code>mcstat.mean_se(...)</code>
<code>TypeError: 'float' object is not callable</code>	<code>n/(n-1)(...)</code> —— 漏了乘号，Python 以为你要调用一个浮点数
<code>NameError: name 'x' is not defined</code>	用了还没定义的东西。今天两次：填空版没填完； <code>mean</code> 被移到了使用它的那一行之后
<code>AttributeError: ... got an unexpected keyword argument 'lable'</code>	参数名拼写错误。这类错永远在运行时才炸，编辑器查不出来
<code>TypeError: can't multiply sequence by non-int of type 'float'</code>	列表乘浮点。「非整数」这个措辞在暗示：乘整数是合法的（重复列表），但那不是你要的意思
<code>ValueError: could not convert string to float: 'z_mid'</code>	读 csv 忘了跳过表头
<code>fatal: Not a git repository</code>	站错目录了。提示符永远告诉你当前在哪

反解错误信息（今天用了三次）：不要只问「它为什么报错」，要问「它到底看见了什么」。

`git add` 是怎么看见 `-m` 的？→ 只可能来自下一行 → 那行没有另起一行 → 回显里 `spec.mdgit` 就是烟枪。

第 2 部分 · 明天 (9/9) 会用到的

2.1 Markdown (写 README)

```
# 一级标题      ## 二级      ### 三级
**粗体**      *斜体*      `行内代码`
- 无序列表项
1. 有序列表项

| 列1 | 列2 |      表格：第二行的 --- 是分隔符，必须有
| --- | --- |
| a | b |

```python
代码块，三个反引号 + 语言名
```

[链接文字](http://...)      ![图片说明](lesson03_flux.png)
```

README 不是流水账，是一份「我验证了什么」的清单。老师三十秒扫一眼，要能看出这个人写代码是带判断的。

只写你验过的数字，不写「高性能」「高精度」这类虚词——每一句都要是你能被追问的。

2.2 画 R 图 (Q3 的正确答案，十分钟)

```
ax.set_xlabel("depth (cm)")
ax.set_ylabel("relative error R")
ax.set_yscale("log")      # R 跨一个多量级，线性轴会把前半段压成贴地的线
```

csv 里 R 那一列已经存好了，`read_flux_csv` 多读一列就能画。

想回答关于某个量的问题，那个量必须出现在图上。你在这张通量图上 R 一次都没出现过，所以它答不了「R 随深度怎么变」。

2.3 白板题⑥ 会用到的两个公式 (纸笔，不写代码)

```
线性插值： S(e) = S[k] + (S[k+1]-S[k]) * (e-E[k])/(E[k+1]-E[k])
log-log :   在 log 空间做同样的事，化简后是 S(e) = S[k] * (e/E[k])**p
```

三个会炸的地方：`math.log(0)` (截面真的会是 0)、`E[0]=0`、`E[k]==E[k+1]` (ENDF 里故意放重复能量点表示跳变)。

2.4 git push

```
git push -u origin main      # 远端已配好，网络通了就这一条
```

第 3 部分 · 数模比赛（9/10-13）大概率会用到的

四天高强度 Python，一定会用到 **numpy** 和 **matplotlib** —— 而那正是你第 5 课立刻要用的两样。比赛结束当天花 20 分钟，写下「这四天用到了哪些项目里还没用过的技巧」，直接接到第 5 课。

3.1 numpy —— 把「循环算」换成「整块算」

```
import numpy as np

a = np.array([1.0, 2.0, 3.0])      # 从列表造数组
np.zeros(20)      np.ones(20)    # 全 0 / 全 1
np.linspace(0, 5, 21)              # 0 到 5 均匀取 21 个点（含两端）
np.arange(0, 5, 0.25)              # 步长 0.25（不含右端）

a.shape      a.dtype      len(a)      # 形状、元素类型

# 📍 向量化：整个数组一起算，不写 for
b = a * 2 + 1      # 每个元素都做
c = np.exp(-a)      # 每个元素取指数
d = a * b          # 逐元素相乘（不是矩阵乘法！矩阵乘是 a @ b）

a.sum()  a.mean()  a.std(ddof=1)  a.max()  a.argmax()
#                               ↑ ddof=1 才是贝塞尔修正的样本标准差，默认 ddof=0 是总体的

a[2]      a[1:4]      a[-1]      # 切片和列表一样
a[a > 2]      # 布尔索引：取出所有大于 2 的元素
np.where(a > 2, a, 0)      # 满足条件取 a，否则取 0
```

numpy 数组和 Python 列表长得像，行为完全不同：

`[1,2] * 3` → `[1,2,1,2,1,2]`（重复）

`np.array([1,2]) * 3` → `array([3,6])`（每个元素乘 3）

所以今天那个「列表不能乘浮点」的问题，换成 numpy 数组就不存在了。

对本项目：第 5 课读 ENDF 截面表、第 6 课往下所有能量网格计算，全部是 numpy 数组。今天的 `mean_se` 仍然用标量累加，因为那是给 GPU 准备的写法。

3.2 matplotlib 进阶（数模的图基本都是这几样）

```
# 多子图：2 行 1 列，共享 x 轴
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7,7), sharex=True)

ax1.plot(x, y1, label="模型")          # 普通折线
ax1.scatter(x, y2, s=12, label="实测") # 散点，s 是点面积
ax1.fill_between(x, lo, hi, alpha=0.2) # 置信带
ax2.bar(names, values)                 # 柱状图
ax2.hist(samples, bins=30)             # 直方图

ax1.set_xlim(0, 5)    ax1.set_ylim(0, 2)    # 手动定范围
ax1.axhline(1.0, ls="--", color="gray")    # 水平参考线
ax1.axvline(2.5, ls=":")                    # 垂直参考线
ax1.annotate("峰值", xy=(0.4, 2.03), xytext=(1, 2.2),
             arrowprops=dict(arrowstyle="->")) # 带箭头的标注

fig.tight_layout()          # 自动排开，防止标签互相压
fig.savefig("fig.png", dpi=300) # 论文插图用 300
```

3.3 pandas（数模读数据几乎必用）

```
import pandas as pd

df = pd.read_csv("data.csv")          # 一行读进来，自动识别表头
df = pd.read_excel("data.xlsx")
df.head()      df.shape      df.columns      df.describe()

df["列名"]          # 取一列 (Series)
df[["列A", "列B"]]  # 取多列
df[df["年份"] > 2020] # 按条件筛行
df.sort_values("列名", ascending=False)
df.groupby("类别")["数值"].mean()      # 分组统计
df.isna().sum()                        # 每列有多少缺失值
df.dropna()      df.fillna(0)          # 丢弃 / 填补缺失

df.to_csv("out.csv", index=False, encoding="utf-8-sig")
#                                     ↑ Excel 打开中文不乱码要用这个编码
df["列名"].values          # 转成 numpy 数组，好画图
```

pandas 没装的话：conda install -c conda-forge pandas。数模常配 scipy（拟合、优化、统计检验）和 scikit-learn（回归、聚类）。

3.4 try / except（读别人给的数据时必用）

```
try:
    x = float(s)
except ValueError:
    x = None          # 转不动就当缺失，不要让整个程序崩掉
```

不要写 except：光秃秃一个——那会把 Ctrl+C 和所有拼写错误一起吞掉，程序会以最难查的方式出错。永远写清你要接住哪一种异常。

3.5 dict (字典)

```
d = {"H": 1.0, "C": 12.0}
d["O"] = 16.0           # 加/改
d["H"]                  # 取；键不存在会 KeyError
d.get("N", 0.0)          # 取不到就返回默认值，不报错
for k, v in d.items():   # 遍历键值对
    "H" in d             # 判断键在不在
```

第 5 课算软组织混合截面时，「元素 → 原子数密度」正是一个 dict。

第 4 部分 · 方法论（比语法值钱）

| 规矩 | 今天的实例 |
|-----------------------|---|
| 一条 assert 要有两个独立算出来的量 | 自测 1 的方差 5/3 是你手算的，不是另一段代码算的 |
| 写完问：什么错误能从它底下溜过去？ | 自测 2 原写法（全等于期望 $\rightarrow \chi^2=0$ ）连「永远返回 0」的实现都拦不住 |
| 判据也会错，而且没人去判判据 | <code>assert n>=1</code> 放行了必然除零的 <code>n=1</code> |
| 判据本身可能根本不能工作 | <code>git diff</code> 对 UTF-16 只会说 "Binary files differ" |
| 判据有分辨率，超出分辨率的东西它看不见 | 打印只到 6 位小数 $\rightarrow$ 看不见第 16 位的浮点重结合；
「输出逐位不变」 $\rightarrow$ 看不见删掉的 120 行注释；
20 层分箱 $\rightarrow$ 看不见峰的真实位置 |
| 重构的判据是白送的 | 旧输出就是标准答案，不需要你设计任何新判据 |
| 一次计算，两个出口 | <code>print_flux_table</code> 里 <code>phi/err/R</code> 只算一遍，同时打印和写 csv |
| 能验的可以外包，要设计判据的不行 | lesson04 的机械改造交给 Claude 是安全的（旧输出会当场戳穿错误）；
新写一课的容差不行 |
| 验的必须是你真正要用的那个东西 | 在 base 环境查到 matplotlib，而脚本跑在 openmc 环境 |
| 一张图只能回答它被设计来回答的问题 | 通量图答不了「R 随深度怎么变」，因为 R 没画在上面 |
| 屏幕上打出来了但没人看，等于没打 | <code>git commit</code> 的文件清单和消息对不上，当场就写着，但没人看 |

第 5 部分 · 速查表（放手边）

git

```
git status                                git log --oneline --stat
git add A B; git commit -m "..."       git diff          git diff --staged
git diff --numstat -- 文件              git show <id>:<文件>
git reset <id>                          git checkout 文件
git remote -v                           git push -u origin main
git commit --amend -m "新说明"          ← 只在没 push 过时用
```

Python

```
import mcstat                             import numpy as np
import matplotlib.pyplot as plt           from math import log, exp, sqrt

def f(a, b=默认值):
    """一句话说明。"""
    assert 条件, "消息 %d" % x
    return x, y, z

if __name__ == "__main__": ...

L[1:]   L[:2]   L[-1]   len(L)   L.append(x)
[e*2 for e in L]           # 列表推导式
"a,b".split(",")          x ".strip()   float("1.5")
"%0.6f" % x               "%.2e" % x    "%-12s" % s    "%+10.2f" % x

with open(p, "r", encoding="utf-8") as f: lines = f.readlines()
with open(p, "w", encoding="utf-8") as f: f.write("...\n")
```

matplotlib

```
fig, ax = plt.subplots(figsize=(7,5))
ax.errorbar(x, y, yerr=e, label="...", marker="o", markersize=3, capsize=2)
ax.set_xlabel("... (unit)"); ax.set_ylabel("... (unit)")
ax.legend(); ax.grid(True, alpha=0.3); ax.set_yscale("log")
fig.savefig("x.png", dpi=150, bbox_inches="tight")      # ← 必须在 show 之前
plt.show()
```

命令行

```
conda activate openmc
conda install -c conda-forge 包名
python -c "import 包; print(包.__version__)"
python -u 脚本.py > out.txt
Compare-Object (Get-Content a.txt) (Get-Content b.txt)      # 无输出 = 相同
python              # 交互式解释器, exit() 退出
```